RHODES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

COMPUTER SCIENCE 302

SOFTWARE DEVELOPMENT PRACTICES and
FUNCTIONAL PROGRAMMING
EXAMINATION: JUNE 2023

Internal Examiner: Dr Y Motara
External examiner: Prof. I Sanders

MARKS: 100
DURATION: 4 hours

GENERAL INSTRUCTIONS TO CANDIDATES

1. This paper consists of 14 pages and 10 questions. ***PLEASE MAKE SURE THAT YOU HAVE A COMPLETE PAPER.***

2. Any use of mutability or forbidden constructs (*when*, *for*, *while*, <-, etc) in a practical answer will result in a mark of 0 for that answer. Do not use built-in ADTs; build them yourself.

3. Answer Sections A and B on this paper. *If insufficient space has been provided for your answers, please use the spare sheets provided at the back of each section.* Think deeply, write clearly.

4. Answer Sections C and D by modifying the skeleton code files provided on the computer, saving your work to the mapped network drive.

5. You may use software on your local computer to help you to answer the questions. You may not access internet resources during the exam.

6. You may refer to the Oxford English Dictionary during the exam. If you do not understand a question, ask the invigilator to clarify requirements.

7. **Save regularly.**

PLEASE DO NOT TURN OVER THIS PAGE UNTIL TOLD TO DO SO.

SECTION A: SOFTWARE DEVELOPMENT PRACTICES [25 Marks]

Question 1: Software ecosystem **[8 Marks]**

(a) Describe two of the typical responsibilities of a package manager. **(2)**

(b) Why would you use a task runner to run other tools, rather than running them yourself? **(1)**

(c) What kinds of issues does a *linter* pick up, and how are they different from the issues that a compiler would pick up? **(1)**

(d) Why would you use a *formatter* in a team environment? **(1)**

(e) What is a *transitive dependency*? **(1)**

(f) What is *prompting*, in the context of using AI tools? **(1)**

(g) At a high level, describe how a LLM GPT model such as Copilot chooses the code suggestions that it gives to you. Your explanation should allow me to understand why Copilot will never choose to suggest `null` as the next word if I write: `int x =` **(1)**

Question 2: Version Control **[12 Marks]**

(a) What is a "remote"? **(1)**

(b) Draw a Git DAG (including branch labels) in which it is necessary to use a *three-way merge* for merging two branches. Your DAG must have at least three nodes on every branch. **(4)**

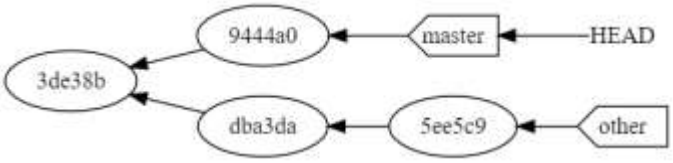
- If you have drawn more than two branches, indicate the branches to be merged by circling the branch labels.
- There are three important nodes in a three-way merge. Mark each of them with an "X".

(c) Can a conflict occur during a *fast-forward merge*? Explain why or why not. **(1)**

(d) Why is it impossible for Git graphs to be *cyclic*? (Do not simply say "because it is acyclic" or "that's what acyclic means". Give me a logical reason for why it cannot be cyclic.) **(1)**

(e) What does Git use the HEAD pointer for? **(1)**

(f) Assume that you have the following Git DAG: **(2)**



Which Git commands would you use to "remove" the last commit on the **other** branch? (If you cannot remember the commands, describe the operations).

(g) Why is *copy-modify-merge* considered to be a more productive way of working than *lock-modify-unlock*, despite being more complex? **(1)**

(h) Why is a distributed version control system resistant to network failure? **(1)**

Question 3: Testing **[5 Marks]**

(a) Describe what happens in each of the Arrange-Act-Assert phases of a unit test. **(3)**

(b) What is the difference between a *unit test* and an *integration test*? **(1)**

(c) Why is the *thorough* testing principle important? **(1)**

SECTION B: FUNCTIONAL PROGRAMMING THEORY [25 Marks]

Question 4 **[13 Marks]**

(a) Consider the following functions. **(3)**

❶ `fun a b -> a - b`

❷ `fun f x -> f x`

❸ `fun c d -> d (d c + ".")`

Fill out the following table with the correct types for each function. Remember that if a part of the domain/codomain is a function, you should place brackets around that part to indicate that.

	Domain	**Codomain**
❶		
❷		
❸		

(b) What is a *lambda function*? **(1)**

(c) Explain why there are no (typical) race conditions in purely functional code. **(1)**

(d) A friend runs their functional program and it crashes due to stack overflow. A tutor advises them to make their recursive functions *tail recursive* to avoid this. Explain what this means, and why it will solve the problem. **(2)**

(e) Write a tail recursive function that does not use an accumulator. If this is impossible, explain why all tail recursive functions need to have an accumulator. **(1)**

(f) What is *currying*? Define the term and write a simple curried function. **(2)**

(g) How does knowledge of a function's domain and codomain help you to understand whether the function is *higher order* or not? **(1)**

(h) Write down one advantage and one disadvantage of *parametric polymorphism*. **(2)**

Question 5 **[6 Marks]**

In typed functional programming, algebraic data types are split into "sum types" and "product types".

(a) Write down a type with a range of 33. **(1)**

(b) Write a pattern that will match the type you create in (a) and bind ***only*** one value to a name. **ONLY** write down the pattern itself, *and nothing else*. **(2)**

(c) The `int * unit * bool` type has the same range as the `int * bool` type. Explain why. **(1)**

(d) Write down a parametrically polymorphic data type that can be used to convert a *partial function* into a *total function*, and explain how it can be used to achieve this. **(2)**

Question 6 **[6 Marks]**

(a) Why is limited expressiveness a good thing in a DSL? **(1)**

(b) Why would you use a *deep* embedding instead of a *shallow* embedding? **(1)**

(c) Describe, in detail, how the `fold` function works. You may use an example and/or diagram to help with your explanation if you feel that it would benefit you. **(2)**

(d) Name and explain the purpose of any other common *higher order* operation that is used in mid-level design. **(2)**

SECTION C: FUNCTIONAL PROGRAMMING [25 Marks]

A correct answer will be awarded the full mark, and an incorrect answer will receive zero; there is no partial marking in this section.

Name your functions as **q7** for Question 7, **q8** for Question 8, and so on. You may create as many helper functions as you want to.

Commented-out tests are provided to help you to create correct answers, and to help you understand the expected inputs and output. However, creating a correct solution that accounts for all cases is ultimately your responsibility, just as it will be if you enter the working world as a software developer!

Question 7 **[5 Marks]**

Write a function that takes an integer as input and returns the number of digits in the integer.

Question 8 **[8 Marks]**

Some languages define a "for" loop that looks like:

```
FOR I := 0 TO 6 DO
  -- body of the loop
END
```

Create a function that implements the logic of "for".

Question 9 **[12 Marks]**

Define a parametrically polymorphic list type named `MyList`. Create:

- A function named `push`, which accepts a value and a list and returns a new list with that value.
- A function named `one`, which accepts a value and creates a list with one value.

Now create a function named `q9` which accepts a value and a list, and places the given value between all elements of the list.

SECTION D: FUNCTIONAL PROBLEM-SOLVING [25 Marks]

Question 10

Your friend, who is a good chess player, decides to make her own version of the game. She calls it "Knights Challenge". One of the spaces on the board is marked with an X. This is the space that the knight must get to. A knight jumps around the board in an "L" shape: two spaces in any direction, and one space to the side. There is only one knight on the board, and the knight starts on the top-left of the board. The knight wins if a sequence of moves gets it to the "X".

Example ("K" = knight):

A	B	C	D	E	F	G	H	
K								8
								7
								6
								5
		X						4
								3
								2
								1

The rows of a chess-board are called *ranks* and numbered 1-8 as above, and the columns are called *files* and are named A-H as above. The knight always starts in A8. The knight cannot cross the edges of the board.

A player can choose to make one of the following moves by putting in the corresponding number:

1. Knight moves to the RIGHT two spaces, then DOWN one space.
2. Knight moves to the RIGHT two spaces, then UP one space.
3. Knight moves to the LEFT two spaces, then DOWN one space.
4. Knight moves to the LEFT two spaces, then UP one space.
5. Knight moves DOWN two spaces, then to the RIGHT one space.
6. Knight moves DOWN two spaces, then to the LEFT one space.
7. Knight moves UP two spaces, then to the RIGHT one space.
8. Knight moves UP two spaces, then to the LEFT one space.

These moves are numbered from 1 to 8, so if a player makes "Move 6", that means that the knight moves DOWN two spaces and then to the LEFT one space. If a move is illegal, then it is ignored.

A skeleton is provided in your exam folder as **Question10**. Your game must pass certain tests, provided with the skeleton, which will use certain functions. The mandatory functions are:

- **init** : a `string * int -> Game` function which represents a new game, with the "X" at the selected spot. If the coordinates of the "X" are outside of the board, then the game is considered to be unwinnable.
- **moves** : a `Game -> int` function that returns the number of legal moves that have been made so far. The count of moves stops just after the knight reaches the "X".
- **go** : an `int -> Game -> Game` function which moves the knight according to the chosen move.
- **status** : a `Game -> string` function that returns "finished", "unfinished", or "unwinnable", depending on the game and whether the knight has visited the "X".
- **knight** : a `Game -> string` function that gives the knight's position. The position is always expressed as a file followed by a rank, e.g. "B7" or "H3".

The `Game` referred to above is a data structure, designed by you, to represent the state of the game. The unit tests included in the skeleton solution will help you to understand all the requirements. Your mark for this question is dependent on the number of included tests which pass.

END OF EXAMINATION

ADDITIONAL WORKING

ADDITIONAL WORKING